IF4020 - Kriptografi

Laporan Tugas 2






Kelompok

Samuel Gerrard H. Girsang	13523064
Nadhif Al Rozin	13523076
Ivant Samuel Silaban	13523129
Joel Hotlan Haris Siahaan	13523025

PROGRAM STUDI TEKNIK INFORMATIKA
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
JL. GANESHA 10, BANDUNG 40132

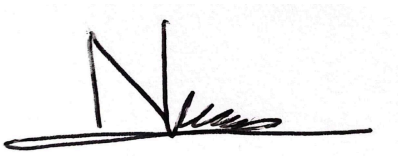
2026

PERNYATAAN TIDAK MELAKUKAN KECURANGAN

Kami menyatakan bahwa kode program yang dihasilkan bukan merupakan hasil salinan mentah (*raw output*) dari *Generative AI*, melainkan hasil pengembangan dan penulisan mandiri.



Samuel Gerrard H. Girsang



Nadhif Al Rozin



Ivant Samuel Silaban



Joel Hotlan Haris Siahaan

PENDAHULUAN

Perkembangan teknologi informasi dan komunikasi sangat pesat di dekade ini. Hal itu menuntut adanya pengamanan data yang bisa diandalkan untuk menjaga kerahasiaan dan integritas informasi dari ancaman siber. Dalam ranah kriptografi simetris, *block cipher* punya peran yang krusial sebagai fondasi utama pengamanan data. Sebagai perbandingan historis dan teoritis, standar kriptografi block cipher yang digunakan adalah Data Encryption Standard (DES) yang menggunakan jaringan feistel 64 bit, meski saat ini sudah dinilai rentan karena ukuran blok dan kunci yang terlalu pendek. Sementara itu, Advanced Encryption Standard (AES) menggunakan *Substitution-Permutation Network* (SPN) dengan ukuran blok 128-bit yang menawarkan performa dan keamanan tinggi, namun memiliki struktur aljabar statis yang sering kali menjadi target analisis diferensial maupun linear tingkat lanjut. Berdasarkan evaluasi terhadap kelemahan sistem pendahulu tersebut, perancangan algoritma *block cipher* baru ini berfokus pada peningkatan ketahanan melalui kombinasi arsitektur yang dinamis dan adaptif.

Gagasan dan pendekatan utama yang digunakan dalam rancangan block cipher baru ini mengadopsi struktur jaringan feistel 128-bit dengan 16 putaran, serta mengintegrasikan operasi ARX (Addition-Rotation-XOR). Untuk menghindari kerentanan terhadap analisis serangan terstruktur yang sering ditemukan pada S-Box statis milik DES atau AES, rancangan ini menerapkan Key-Dependent S-Box yang dibangkitkan secara dinamis berbasis kunci master. Selain itu, penyertaan konstanta universal irasional (Golden Ratio dan fraksi π) dalam pembangkitan subkunci bertujuan untuk mengeliminasi simetri putaran serta memaksimalkan *avalanche effect* sejak putaran-putaran awal.

DASAR TEORI

1. **Confusion & Diffusion**

   Confusion adalah pengaburan korelasi lokal antara plaintext dan ciphertext dengan melakukan variasi aplikasi kunci ke data. Sedangkan diffusion adalah menyembunyikan statistik plaintext dengan menyebarkannya ke area yang luas pada ciphertext. Meskipun sebuah cipher bisa hanya menggunakan salah satu, tapi cipher yang baik akan menggunakan keduanya

2. **Block cipher**

   Block cipher adalah algoritma kriptografi simetris yang mengenkripsi dan mendekripsi data dengan cara memprosesnya dalam potongan-potongan (blok) berukuran tetap dengan kunci yang sama. Data asli (plaintext) dipecah menjadi blok-blok dengan ukuran yang sama, umumnya berukuran 64 bit atau 128 bit. Cipher akan merubah blok plaintext menjadi blok ciphertext melalui beberapa putaran (rounds) operasi matematis dan logis (seperti substitusi dan permutasi).

3. **ECB, CBC, CFB, OFB & CTR**

   Pada blok cipher, karena ukuran blok yang terbatas (seringkali lebih kecil dari panjang plaintext). Plaintext yang lebih panjang akan diproses dengan mode mode operasi untuk mengaturnya:

   - Electronic Code Book (ECB)

     Enkripsi: $C_i = E_K(P_i)$
     Dekripsi: $P_i = D_k(C_i)$

     Setiap blok plaintext dienkripsi secara mandiri dan terpisah menggunakan kunci yang sama, blok 1 tidak mempengaruhi blok 2. Sangat cepat, dan bisa parallel, kesalahan di 1 bagian tidak akan membuat bagian lain salah. Sayangnya jauh lebih mudah ditembus karena untuk data yang berukuran besar, blok dengan plaintext sama hasil ciphertextnya akan sama pula

   - Cipher Block Chaining (CBC)

     Enkripsi: $C_i = E_K(P_i \oplus C_{i-1})$
     Dekripsi: $P_i = D_K(C_i) \oplus C_{i-1}$
     Jika $i = 0, C_{i-1} = IV$

     Mode ini mulai ada konsep ketergantungan antar blok, dilakukan dengan metode XOR. Sebelum dienkripsi, blok plaintext akan di-XOR terlebih dahulu dengan hasil enkripsi dari blok sebelumnya. Blok pertama menggunakan IV (Initialization Vector) sebagai pemicu awal. Karena ketergantungan, paralel

tidak dimungkinkan, kesalahan di 1 bagian akan membuat bagian selanjutnya rusak. Sama halnya dengan blok plaintext yang sama, hasil ciphertext nya akan berbeda

- Cipher Feedback (CFB)

  Enkripsi: $C_i = P_i \oplus E_K(C_{i-1})$
  Dekripsi: $P_i = C_i \oplus E_K(C_{i-1})$
  Jika $i = 0$, $C_{i-1} = IV$

  Mode ini mengubah block cipher menjadi seolah-olah bekerja seperti stream cipher self-synchronizing. Yang dienkripsi oleh algoritma bukanlah plaintext-nya, melainkan ciphertext dari blok sebelumnya (atau IV untuk blok pertama). Hasil enkripsi tersebut baru di-XOR dengan plaintext.

- Output Feedback (OFB)

  Enkripsi: $C_i = P_i \oplus O_i$
  Dekripsi: $P_i = C_i \oplus O_i$
  $O_i = E_K(O_{i-1})$. Jika $i = 0$, $E_K(IV)$

  Mode ini mengubah block cipher menjadi synchronous stream cipher. Algoritma kriptografi menghasilkan aliran kunci (keystream) O_i secara independen dari plaintext maupun ciphertext. Keystream ini kemudian di-XOR dengan plaintext. Karena pembentukan keystream tidak bergantung pada pesan, proses pre-computation keystream bisa dilakukan sebelum data diterima. Kerusakan bit pada ciphertext saat transmisi hanya merusak bit yang bersangkutan pada plaintext (tidak meluas ke blok lain).

- Counter (CTR)

  Enkripsi: $C_i = P_i \oplus E_K(Nonce \,||\, Counter_i)$
  Dekripsi: $P_i = C_i \oplus E_K(Nonce \,||\, Counter_i)$

  Mode CTR memproses data sebagai stream cipher dengan mengenkripsi gabungan nilai unik sekali pakai ($Nonce$) dan nilai pencacah ($Counter$) yang meningkat untuk setiap blok.
  - Paralelisasi Penuh: Berbeda dengan CBC dan OFB, setiap blok dalam mode CTR dapat di enkripsi atau dekripsi secara independen dan

bersamaan (paralel), menjadikannya sangat cepat pada perangkat lunak maupun perangkat keras modern.
- Random Access: Memungkinkan akses acak untuk dekripsi blok manapun tanpa perlu memproses blok sebelumnya.

4. **Skema padding**

Block cipher membutuhkan input data dengan ukuran kelipatan blok tetap (misalnya 64 bit atau 128 bit). Jika ukuran plaintext tidak habis dibagi ukuran blok, skema padding digunakan untuk menambah bit ekstra pada blok terakhir.

- PKCS#7 / PKCS#5: Menambahkan N byte yang masing-masing bernilai N (dalam format heksadesimal/byte). Contoh: Jika tersisa 3 byte kosong pada blok, maka ditambahkan byte 0x03 0x03 0x03. Jika panjang plaintext sudah pas dengan kelipatan blok, satu blok penuh berisi padding (misal 16 byte bernilai 0x10) tetap ditambahkan agar dekriptor tahu ada padding yang harus dilepas.
- ISO/IEC 7816-4 (Bit Padding): Menambahkan bit 1 diikuti oleh sejumlah bit 0 hingga mencapai batas ukuran blok (1000...00).
- Zero Padding: Menambahkan byte bernilai 0x00 hingga blok terpenuhi. Memiliki kelemahan jika plaintext asli diakhiri dengan byte 0x00, karena sistem sulit membedakan antara data asli dan padding saat dekripsi.

5. **Avalanche effect**

Avalanche effect (efek longsoran) adalah sifat ideal dari algoritma kriptografi di mana perubahan kecil pada input (baik pada plaintext maupun pada kunci) akan menghasilkan perubahan yang drastis pada ciphertext.

- Prinsip Utama: Perubahan 1 bit saja pada plaintext atau kunci idealnya merubah sekitar 50% bit pada ciphertext secara acak.
- Pentingnya Kriptografi: Sifat ini menjamin terpenuhinya prinsip diffusion dan confusion. Jika suatu algoritma tidak memiliki avalanche effect yang kuat, penyerang dapat menggunakan analisis diferensial untuk memprediksi struktur kunci atau plaintext berdasarkan pola perubahan bit ciphertext.

PERANCANGAN BLOCK CIPHER

1. Ringkasan

Algoritma *block cipher* yang kami rancang adalah cipher simetris yang beroperasi dalam blok data sebesar 128 bit menggunakan kunci master berukuran 128 bit juga. Kami mengadopsi model Jaringan Feistel untuk algoritma kami dengan 16 putaran (rounds).

Tidak lupa juga kami memadukan prinsip dasar kriptografi Shannon, yaitu *confusion* dan *difusi*, melalui kombinasi operasi ARX (Addition-Rotation-XOR) serta subtitusi non-linear menggunakan *key-dependent S-box*.

Ringkasan arsitektur dapat dilihat dari tabel berikut:

Tabel 3.1.1 Ringkasan Parameter Algoritma

Parameter	**Spesifikasi**
Ukuran Blok Data	128 bit
Panjang Master Key	128 bit
Jumlah Putaran (Rounds)	16 putaran
Arsitektur Cipher	Fesitel Network
Panjang *Subkey*	64 bit per putaran
S-Box	8 x 8 bit (256 entries)
Operasi Tambahan	ARX + Subtitusi Byte
Skema Padding	PKCS#7

2. Arsitektur Umum Jaringan Feistel

Algoritma menggunakan topologi Jaringan Feistel. Pada setiap putaran, 128-bit dipartisi secara simetris menjadi upa kiri (L) berukuran 64 bit dan upa kanan (R) berukuran 64-bit:

$$Plaintext\ P\ =\ (L_0,\ R_0),\ |L_0|\ =\ 64\ bit,\ |R_0|\ =\ 64\ bit$$

2.1. Mekanisme Putaran Feistel

Pada setiap putaran r (r=0,1, … ,15) transformasi dilakukan menurut relasi:

$$L_{r+1}\ =\ R_r$$

$$R_{r+1}\ =\ L_r \otimes F(R_r,\ K_r, S)$$

Di mana:

F = fungsi putaran

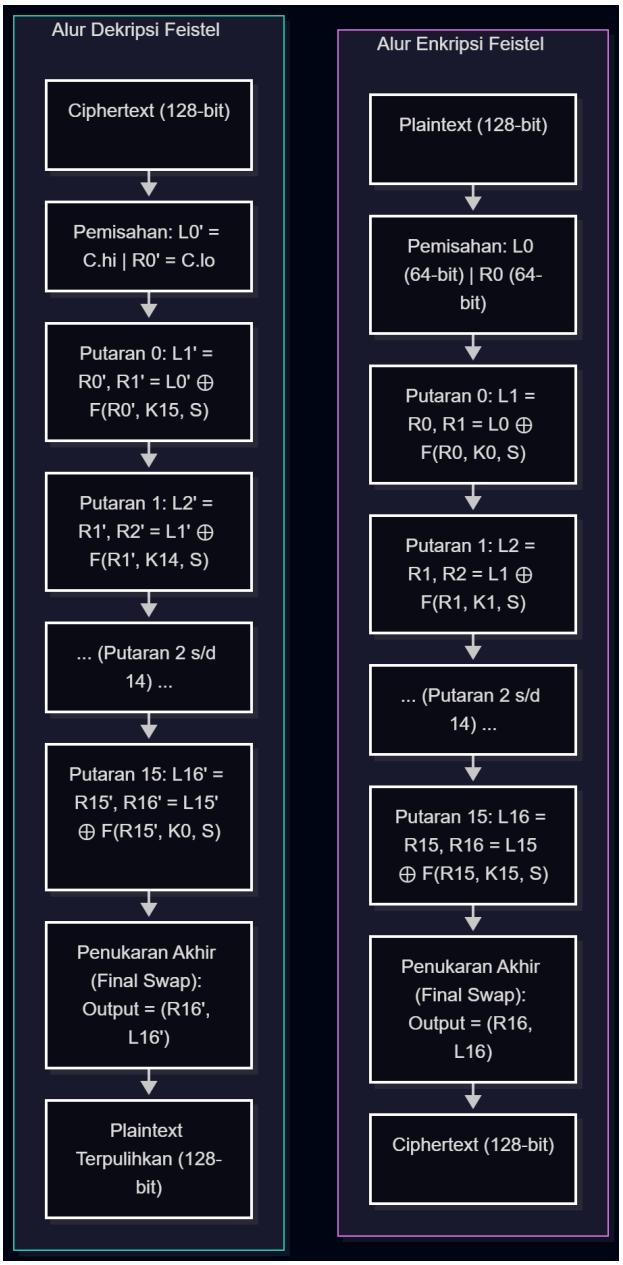
Kr = subkunci putaran ke-r (64 bit)

S = Kotak-S dinamis 256-byte

Setelah putaran ke-16 selesai, dilakukan operasi penukaran akhir untuk menghasilkan ciphertext:

$$Ciphertext\ C\ =\ (R_{16},\ L_{16})$$

2.2. Diagram Alur Arsitektur Feistel



Gambar 3.2.2.1. Diagram Alur Arsitektur Feistel

Salah satu keunggulan arsitektur Feistel adalah sifat invertibilitas inheren, dimana fungsi putaran F tidak dituntut memiliki fungsi invers matematis (F^-1), karena dekripsi dicapai hanya dengan membalik urutan subkunci (K15->K0).

3. Skema Key Generation

Skema *Key Expansion* bertugas mentransformasikan kunci mater 128-bit menjadi:

1. Key-Dependent S-Box -> array 256 byte unik.
2. 16 Subkunci Putaran -> masing-masing berukuran 64 bit (K_0, K_1, …, K_15)

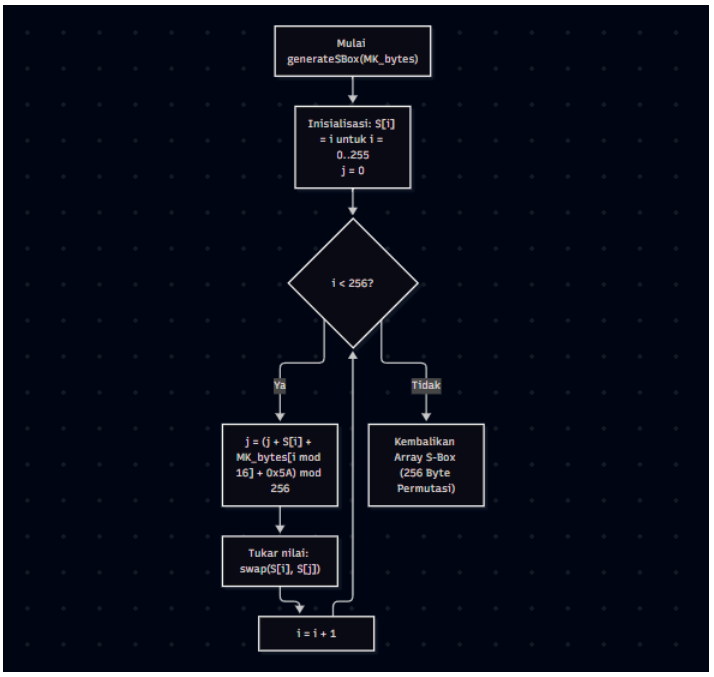
3.1. Pembangkitan Kotak-S Dinamis

Daripada menggunakan Kotak-S statis (seperti yang dipakai di AES dan DES) yang rentan terhadap analisis serangan terstruktur jika terdapat kelemahan aljabar, cipher kami mengimplementasikan Kotak-S yang diturunkan secara dinamis dari kunci master melalui algoritma pengacakan Fisher-Yates / RC4 KSA (*Key-Scheduling Algorithm*)

Algoritma Pembangkitan S-box:

1. Inisialisasi array subtitusi dengan permutasi identitas:
$$S[i] = i, \forall i \in \{0, 1, \ldots, 255\}$$
2. Inisialisasi akumulator indeks: $j = 0$ (modulo 256).
3. Untuk $i = 0$ hingga 255:
$$j = (j + S[i] + MK_{bytes}[i \ (mod\ 16)] + 0 \times 5A) \ (mod\ 256)$$
$$swap(S[i], S[j])$$

Konstanta $0 \times 5A$ (01011010_2) berfungsi sebagai perturbasi *bitwise* untuk mencegah degradasi entropi ketika kunci master mengandung deretan byte nol (0 x 00). Hasil akhir S dijamin merupakan pemetaan bijektif yaitu setiap elemen 0 x 00 hingga 0 x FF muncul tepat satu kali.



Gambar 3.3.1.1 Diagram Alur Pembangkitan S-box

3.2. Pembangkitan 16 Subkunci 64-Bit

Kunci master 128-bit dipecah menjadi dua register status 64-bit:

$$k_0 = MK.hi$$
$$k_1 = MK.lo$$

Untuk menghasilkan 16 subkunci yang independen, memiliki difusi bit yang tinggi dan kebal terhadap slide attacks, digunakan pembangkit iteratif non-linear yang mengintegrasikan konstanta universal irasional (*nothing-up-my-sleeve-numbersi*).

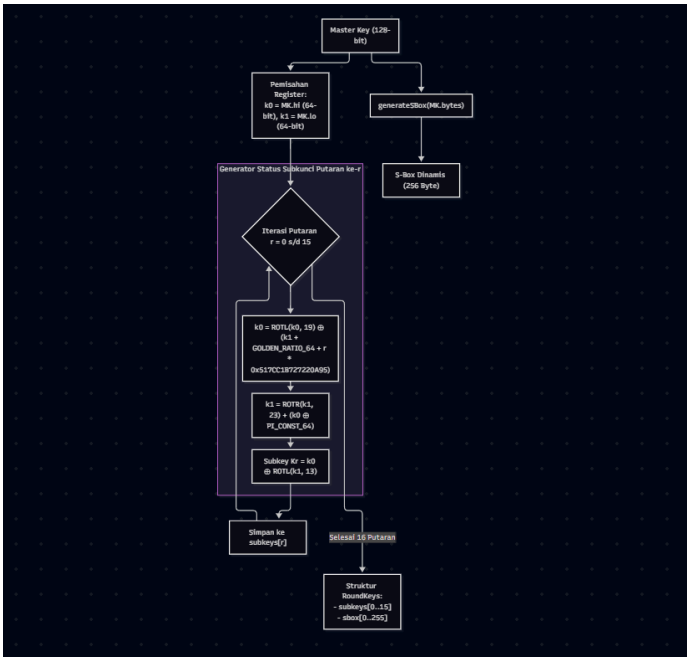
Tabel 3.3.2.1 Konstanta Kriptografi yang Digunakan

Konstanta	**Nilai Heksadesimal**	**Keterangan**
GOLDEN_RATIO_64	0x9E3779B97F4A7C15	$\lfloor 2^{64} \times (\frac{\sqrt{5}-1}{2}) \rfloor$ (Golden Ratio, fraksi irasional maksimal)
PI_CONST_64	0x243F6A8885A308D3	$\lfloor 2^{64} \times (\pi - 3) \rfloor$ (Fraksi heksadesimal bilangan π)
ROUND_CONST_MULT	0x517CC1B727220A95	Pengali bilangan ganjil berukuran 64-bit untuk eliminasi simetri putaran

Persamaan Rekurensi untuk putaran $r = 0, 1, \ldots, 15$:

1. Pembaruan register k0:
$$k_0 \leftarrow ROTL_{64}(k_0, 19) \oplus (k_1 + GOLDEN_RATIO_64 + (r \times 0 \times 517CC1B727220A95)) \ (mod\ 2^{64})$$
2. Pembaruan register k1:
$$k1 \leftarrow (ROTR_{64}(k_1, 23) + (k_0 \oplus PI_CONST_64)) \ (mod\ 2^{64})$$
3. Penurunan subkunci putaran ke-r:
$$K_r = k_0 \oplus ROTL_{64}(k_1, 13)$$

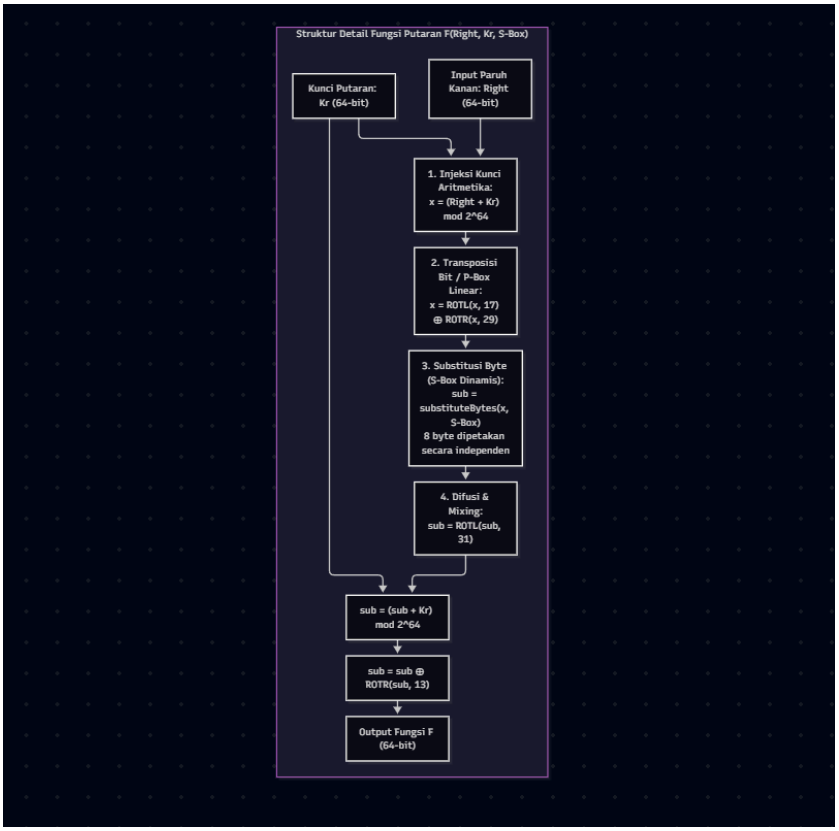
3.3. Diagram Skema Pembentukan Kunci Putaran



Gambar 3.3.1 Diagram Skema Pembentukan Kunci Putaran

4. Perancangan Komponen Primitif Algoritma

Fungsi putaran dirancang menggunakan empat lapisan transformasi komplementer: Injeksi Kunci, Transposisi Bit, Subtitusi Non-linear Byte (Kotak - S), dan Difusi Non-linear Akhir.



Gambar 3.4.1 Struktur Detail Fungsi Putaran F(Right, kr, S-Box)

4.1. Injeksi Kunci Aritmetika

Operasi pertama dalam fungsi putaran adalah pencampuran paruh kanan dengan subkunci putaran menggunakan penjumlahan modulo 2^{64}:

$$x = addMod64(R, K_c) = (R + K_c) \, (mod \, 2^{64})$$

Penjumlahan modular memicu perambatan pembawa (carry bits) dari bit berorde rendah (LSB) ke bit berorde tinggi (MSB). Hal ini memberikan sifat non-linearitas aljabar terhadap operasi XOR bitwise yang digunakan pada tahap Feistel.

4.2. Komponen Permutasi dan Transposisi Bit

Setelah injeksi kunci, data 64-bit dikenakan transposisi bit menggunakan kombinasi rotasi bit melingkar ganda:

$$x = bitTransposition(x) = ROTL_{64}(x, 17) \oplus ROTR_{64}(x, 29)$$

Nilai pergeseran 17 dan 29 dipilih karena saling prima satu sama lain dan prima terhadap ukuran word 64 bit $(gcd(17, 64) = 1, \, gcd(29, 64) = 1, \, gcd(17, 29) = 1)$

Operasi ini memecah batas-batas byte (*byte boundaries*) sehingga satu bit pada posisi tertentu akan disebarkan melintasi beberapa byte sebelum memasuki tahap substitusi S-box.

4.3. Kotak-S & Subtitusi Byte

Komponen substitusi bertindak sebagai sumber utama *confusion* dan resistensi non-linear. Kata 64-bit x dipartisi menjadi 8 byte terpisah $(b_0, b_1, ..., b_7)$:

$$b_i = (x \gg (8 \cdot i)) \, \& \, 0xFF, \; \forall i \in \{0, 1, ..., 7\}$$

Setiap byte kemudian dipetakan ke nilai pengganti melalui Kotak-S dinamis S:

$$s_i = S[b_i]$$

Hasil substitusi digabungkan kembali menjadi kata 64-bit sub:

$$sub = \bigoplus_{i=0}^{7} \left(s_i \ll (8 \cdot i) \right)$$

4.4. Difusi Non-Linier dan Pencampuran Akhir

Tahap akhir fungsi putaran menyebarkan perubahan byte subtitusi ke seluruh register 64-bit melalui kombinasi pergeseran dan penguncian ulang

1. Rotasi Cepat

$$sub \leftarrow ROTL_{64}(sub, 31)$$

2. Injeksi Ulang Subkunci

$$sub \leftarrow (sub + K_r) \, (mod \, 2^{64})$$

3. Penyebaran Silang XOR-Rotasi

$$sub \leftarrow sub \oplus ROTR_{64}(sub, 13)$$

Penggunaan rotasi 31 dan 13 memastikan dispersi bit dari MSB ke LSB dan sebaliknya berlangsung secara optimal sebelum hasil fungsi putaran di-XOR ke upa kiri (L)

5. Rancangan Rinci Alur Enkripsi Blok

5.1. Spesifikasi Input dan Output

Inputnya adalah Blok Plaintext P (128 bit) dan Struktur RoundKeys (16 subkunci 64-bit + S-box). Sedangkan outputnya adalah Blok Ciphertext C (128 bit)

5.2. Tahapan Algoritma Enkripsi Blok

1. Pemisahan Blok Plaintext

$$L_0 = P.hi$$
$$R_0 = P.lo$$

2. Iterasi 16 Putaran Feistel

$$temp = L_r \oplus F(R_r, K_r, S)$$
$$L_{r+1} = R_r$$
$$R_{r+1} = temp$$

3. Final Swap

$$C.hi = R_{16}$$
$$C.lo = L_{16}$$
$$Ciphertext\ C = (R_{16}, L_{16})$$

6. Rancangan Rinci Alur Dekripsi Blok

6.1. Spesifikasi Input dan Output

Inputnya adalah Blok Ciphertext C (128 bit) dan Struktur RoundKeys. Dan outputnya adalah Blok Plaintext P (128 bit).

6.2. Tahapan Algoritma Dekripsi Blok

Dekripsi Feistel memanfaatkan struktur identik dengan alur enkripsi, namun menerapkan subkunci dalam urutan terbalik:

$$K_{15}, K_{14}, K_{13}, \ldots, K_0$$

1. Pemisahan Blok Ciphertext

$$L'_0 = C.hi = R_{16}$$
$$R'_0 = C.lo = L_{16}$$

2. Iterasi 16 Putaran Feistel Terbalik

$$temp = L_{inv} \oplus F(R_{inv}, K_r, S)$$
$$L_{inv+1} = R_{inv}$$

$$R_{inv+1} = temp$$

3. Final Swap

$$P.hi = R'_{16}$$
$$P.lo = L'_{16}$$
$$Plaintext\ Terpulihkan\ P = (R'_{16}, L'_{16})$$

6.3. Pembuktian Matematis Kebenaran Dekripsi Feistel

Pada akhir enkripsi putaran ke-16:

$$L_{16} = R_{15}$$
$$R_{16} = L_{15} \oplus F(R_{15}, K_{15}, S)$$

Ciphertext yang dihasilkan adalah $C = (R_{15}, L_{16})$

Pada awal dekripsi:

$$L'_0 = R_{16} = L_{15} \oplus (R_{15}, K_{15}, S)$$
$$R'_0 = L_{16} = R_{15}$$

Pada putaran pertama dekripsi mengugnakan

$$temp = L'_0 \oplus F(R'_0, K_{15}, S)$$
$$temp = [L_{15} \oplus F(R_{15}, K_{15}, S)] \oplus F(R_{15}, K_{15}, S)$$

Karena sifat involusi XOR ($A \oplus B \oplus B = A$):

$$temp = L_{16}$$

Sehingga register baru adalah

$$L'_1 = R'_0 = R_{16}$$
$$R'_1 = temp = L_{15}$$

Nilai status (L'_1, R'_1) persis setara dengan status sebelum putaran ke-16 pada data enkripsi. Melalui induksi matematik, proses ini berjalan mundur hingga putarna r = 0, memulihkan (L_0, R_0) secara sempurna

7. Skema Penanganan Data dan Padding

Karena cipher beroperasi pada blok tetap 16 byte (128 bit), data input dengan panjang sembarang harus diselaraskan.

a. Formulasi Padding

Panjang byte tambahan dihitung dengan:

Data diperluas dengan menambahkan byte $padLen$ sebanyak $padLen$ kali. Jika data berukuran 11 byte, maka tambahkan 5 byte bernilai 0 x 65. Jika data merupakan kelipatan tepat 16 byte, tambahkan 1 blok penuh (16 byte bernilai 0 x 10).

b. Verifikasi Unpadding Aman (*Constant-Time Verification*)

Untuk mencegah kebocoran padding informasi melalui serangan *Padding Oracle*, verifikasi integritas padding byte dilakukan menggunakan akumulator *bitwise difference*:

```
std::uint8_t diff = 0;
for (std::size_t i = data.size() - padLen; i <
data.size(); ++i) {
    diff |= (data[i] ^ padLen);
}
if (diff != 0) throw std::runtime_error("Corrupted
PKCS#7 padding bytes");
```

Jika ditemukan ketidakcocokan nilai byte padding pada interval akhir, `diff != 0` dan sistem menolak blok tersebut.

8. Analisis Keamanan dan Karakteristik Perancangan

1. Efek Longsoran (Avalanche)

Kombinasi rotasi coprime (17, 29, 31, 13) dan penambahan modular $mod\ 2^{64}$ menjamin bahwa perubahan 1 bit pada plaintext atau kunci akan menyebar ke lebih dari 50% bit ciphertext dalam beberapa putaran awal.

2. Resistensi terhadap Serangan Diferensial & Linear

Karena *S-box* di-generate secara dinamis untuk setiap kunci master, penyerang tidak dapat melakukan pra-komputasi tabel distribusi diferensial (*Differential Distribution Table* / DDT) maupun aproksimasi linear (*Linear Approximation Table* / LAT) yang statis.

3. Pencegahan *Weak Keys*

Penjadwalan subkinci menyertakan konstanta universal GOLDEN_RATIO_64 dan PI_CONST_64 yabng mencegah fenomena simetri register atau kunci invarian (*fixed points*).

4. Efisiensi Perangkat Lunak 64-Bit

Semua operasi dasar (rotasi, penjumlahan, XOR) dipetakan langsung ke instruksi mesin prosesor x86-64 / ARM64 modern (ROL, ROR, ADD, XOR), menghasilkan kinerja *execution* yang optimal tanpa dependensi tabel lookup memori berukuran besar

IMPLEMENTASI & SIMULASI

Deskripsi ringkas antarmuka/program, hasil uji coba enkripsi dan dekripsi pada berkas teks dan biner untuk seluruh mode operasi (ECB, CBC, CFB, OFB, CTR), serta analisis bukti penanganan padding dan verifikasi integritas data.

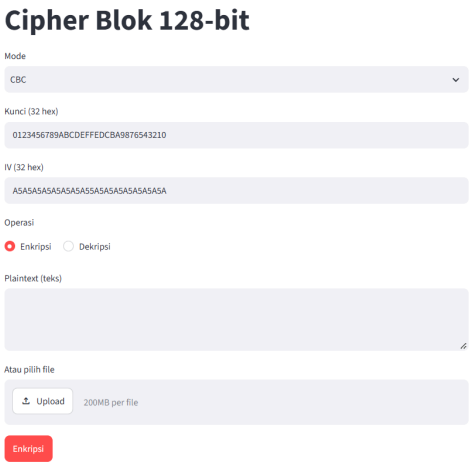
1. Deskripsi Antarmuka Program

Antarmuka dibangun dengan Python dan Streamlit, yang berjalan di web melalui localhost. Antarmuka memanggil implementasi C++ yang telah ada. Beberapa berkas gui serta fungsinya ada pada tabel berikut.

Berkas	Fungsi
src/gui/app.py	Tampilan antarmuka meliputi form input, validasi, dan tampilan hasil
src/gui/cipher.py	Memuat pustaka cipher.dll dengan ctypes, dan menyediakan fungsi encrypt() dan decrypt()
src/gui/cipher_app.cpp	Membungkus encryptBuffer dan decryptBuffer menjadi fungsi C sederhana.

Pada Antarmuka ini terdapat beberapa masukkan yang didukung seperti:

- Mode operasi seperti ECB, CBC, CFB, OFC, atau CTR
- Kunci 128 bit yang ditulis sebagai 32 karakter heksadesimal
- IV 128 bit, ditulis sebagai 32 karakter heksadesimal
- Operasi yaitu enkripsi atau dekripsi
- Data masukkan dapat berupa plain text, cipher text, atau berkas yang diunggah



Program menghasilkan keluaran sebagai teks hasil dekripsi maupun enkripsi jika berupa UTF-8 yang valid, dan sebagai heksadesimal jika tidak. Terdapat fungsionalitas tombol simpan hasil yang akan mengunduh hasil sebagai byte mentah. Program juga menampilkan pesan galat jika kunci atau IV bukan 16 byte heksadesimal, jika panjang ciphertext ECB/CBC bukan kelipatan 16 byte, atau jika padding hasil dekripsi tidak valid.

2. **Hasil Uji Coba**

Parameter yang digunakan

- Kunci: 0123456789ABCDEFFEDCBA9876543210
- IV: A5A5A5A5A5A5A5A55A5A5A5A5A5A5A5A

Setiap berkas dienkripsi lalu ciphertext-nya didekripsi kembali. Keutuhan hasil diverifikasi dengan membandingkan hash SHA-256 berkas asli dan berkas hasil dekripsi.

A. Berkas Teks

> Bahwa sesungguhnya kemerdekaan itu ialah hak segala bangsa dan oleh sebab itu, maka penjajahan di atas dunia harus dihapuskan, karena tidak sesuai dengan perike

B. Berkas Biner

> 000102030405060708090A0B0C0D0E0F10111213141516171819 1A1B1C1D1E1F202122232425262728292A2B2C2D2E2F30313233343536373839 3A3B3C3D3E3F404142434445464748494A4B4C4D4E4F5051525354555 65758595A5B5C5D5E5F606162636465666768696A6B6C6D6E6F707172 737475767778797A7B7C7D7E7F808182838485868788898A8B8C8D8E8 F909192939495969798999A9B9C9D9E9FA0A1A2A3A4A5A6A7A8A9A AABACADAEAFB0B1B2B3B4B5B6B7B8B9BABBBCBDBEBFC0C1C2 C3C4C5C6C7C8C9CACBCCCDCECFD0D1D2D3D4D5D6D7D8D9DAD BDCDDDEDFE0E1E2E3E4E5E6E7E8E9EAEBECEDEEEFF0F1F2F3F4F 5F6F7F8F9FAFBFCFDFEFF00010203

3. **Analisis**

   a. **Penanganan Padding**

	Plaintext	Panjang (bit)
Text	Bahwa sesungguhnya kemerdekaan itu ialah hak segala bangsa dan oleh sebab itu, maka penjajahan di atas dunia harus dihapuskan, karena tidak sesuai dengan perike	1280
Biner(Hex)	000102030405060708090A0B0C0D0E0F10111213141516171819 1A1B1C1D1E1F202122232425262728292A2B2C2D2E2F30313233 3435363738393A3B3C3D3E3F404142434445464748494A4B4C4D 4E4F505152535455565758595A5B5C5D5E5F60616263646566676 8696A6B6C6D6E6F707172737475767778797A7B7C7D7E7F8081 828384858687888 98A8B8C8D8E8F909192939495969798999A9B 9C9D9E9FA0A1A2A3A4A5A6A7A8A9AAABACADAEAFB0B1 B2B3B4B5B6B7B8B9BABBBCBDBEBFC0C1C2C3C4C5C6C7C 8C9CACBCCCDCECFD0D1D2D3D4D5D6D7D8D9DADBDCDD	2080

	DEDFE0E1E2E3E4E5E6E7E8E9EAEBECEDEEEFF0F1F2F3F4F5F6F7F8F9FAFBFCFDFEFF00010203	

	Ciphertext	Panjang (bit)
Text	D4CEFD23D85B801602E4912303D38723F46C78D251CEEF6A428A0224B05401118FBA3296AD0B1FBAFD84568C456B764B4FC3482B51B39F9C54E1D515A907B17A36E62F4BC44DAF2CBCDEFD6D3B773A7A153CCA904FDDC741E8BAA38A2DAF33933C2C0EAA50B99AD569B3632E864D9584356396E769DDFA33DDD746D29236883E071DB61843FBDEE939D0AC9FBE55A3864EA04B2BB6A49BFECF74DA10DDD83B8CE64CC9C00A0B1DBCC74FA8BE8058D6BE	1408
Biner	A27CADCDBF706D7031A4B316B350E9A9C90B0519BE7983115FCE313B0D9A519D9DA338C7B9740E26FB3A601C3DE144979136BE21D1A3E5A9B2E0F0F930E132D4401B04F670176818C1069BB67033E5DCDCE933CED65E3EED16ACFFD05C8AFF25DE7515DF88E0692B3880D63513E6FAE6AED092022C7E96E5DC544110E22DE356CD119B85C687866F7EF80F42EC6D70EDA7638C17FF788BC52B071427C52413955BCFEF84A4A2DA048EEB6181B43D848D1CE14F0CBB9D8B013692CA774598C5D986DA335B97B28513BC004CA27CAD8207F17A2100B48B23F46ECD4FA3F5E5CE589F2CC2A489F99161FC488D69D3F901A581ED34192DC6FC8748E8BC037554F60830F35A874C54113CC52DF3BA5A56FB4D	2176

Dapat dilihat dari plaintext Text dan Biner. Panjang bit text habis dibagi 128 (menjadi 10 blok) sehingga ditambahkan 1 blok penuh berisi padding (sesuai dengan aturan PKCS#7). Dapat dilihat pada ciphertextnya yang 1408 bit (11 blok).

Sedangkan untuk plaintext Biner. Panjang bitnya adalah 2080 (16.25) sehingga ditambahkan padding yang menyebabkan ciphertextnya sepanjang 2176 (17 blok)

b. Integritas Data

Text EBC

		SHA-256
Plaintext Awal	Bahwa sesungguhnya kemerdekaan itu ialah hak segala bangsa dan oleh sebab itu, maka penjajahan di atas dunia harus dihapuskan, karena tidak sesuai dengan perike	d1af69fa388f38992f4211d7dcc8b5f9d393226a986349e4407f1fc48df98d9c
Plaintext Akhir	Bahwa sesungguhnya kemerdekaan itu ialah hak segala	d1af69fa388f38992f4211d7d

	bangsa dan oleh sebab itu, maka penjajahan di atas dunia harus dihapuskan, karena tidak sesuai dengan perike	cc8b5f9d393226a986349e4407f1fc48df98d9c

Dapat dilihat jika hasil dekripsi dari plaintext Text yang di enkripsi dengan mode EBC sama dengan plaintext awal, yang bisa dicek lewat SHA-256 nya yang tidak mengalami perubahan. Untuk Text dengan mode lain dan Biner dengan mode lainnya bisa dicek di tabel dibawah:

				SHA-256
Text	CBC	Plaintext Awal	Bahwa sesungguhnya kemerdekaan itu ialah hak segala bangsa dan oleh sebab itu, maka penjajahan di atas dunia harus dihapuskan, karena tidak sesuai dengan perike	d1af69fa388f38992f4211d7dcc8b5f9d393226a986349e4407f1fc48df98d9c
		Plaintext Akhir	Bahwa sesungguhnya kemerdekaan itu ialah hak segala bangsa dan oleh sebab itu, maka penjajahan di atas dunia harus dihapuskan, karena tidak sesuai dengan perike	d1af69fa388f38992f4211d7dcc8b5f9d393226a986349e4407f1fc48df98d9c
	CFB	Plaintext Awal	Bahwa sesungguhnya kemerdekaan itu ialah hak segala bangsa dan oleh sebab itu, maka penjajahan di atas dunia harus dihapuskan, karena tidak sesuai dengan perike	d1af69fa388f38992f4211d7dcc8b5f9d393226a986349e4407f1fc48df98d9c
		Plaintext Akhir	Bahwa sesungguhnya kemerdekaan itu ialah hak segala bangsa dan oleh sebab itu, maka penjajahan di atas dunia harus dihapuskan, karena tidak sesuai dengan perike	d1af69fa388f38992f4211d7dcc8b5f9d393226a986349e4407f1fc48df98d9c
	OFB	Plaintext Awal	Bahwa sesungguhnya kemerdekaan itu ialah hak segala bangsa dan oleh sebab itu, maka penjajahan di atas dunia harus dihapuskan, karena tidak sesuai dengan perike	d1af69fa388f38992f4211d7dcc8b5f9d393226a986349e4407f1fc48df98d9c
		Plaintext Akhir	Bahwa sesungguhnya kemerdekaan itu ialah hak segala bangsa dan oleh sebab itu, maka penjajahan di atas dunia harus dihapuskan, karena tidak sesuai dengan perike	d1af69fa388f38992f4211d7dcc8b5f9d393226a986349e4407f1fc48df98d9c
	CTR	Plaintext Awal	Bahwa sesungguhnya kemerdekaan itu ialah hak segala bangsa dan oleh sebab	d1af69fa388f38992f4211d7dcc8b5f9d393226a986349e4

			itu, maka penjajahan di atas dunia harus dihapuskan, karena tidak sesuai dengan perike	407f1fc48df98d9c
		Plaintext Akhir	Bahwa sesungguhnya kemerdekaan itu ialah hak segala bangsa dan oleh sebab itu, maka penjajahan di atas dunia harus dihapuskan, karena tidak sesuai dengan perike	d1af69fa388f38992f4211d7dcc8b5f9d393226a986349e4407f1fc48df98d9c

				SHA-256
Biner	EBC	Plaintext Awal	000102030405060708090A0B0C0D0E0F101112131415161718191A1B1C1D1E1F202122232425262728292A2B2C2D2E2F303132333435363738393A3B3C3D3E3F404142434445464748494A4B4C4D4E4F505152535455565758595A5B5C5D5E5F606162636465666768696A6B6C6D6E6F707172737475767778797A7B7C7D7E7F808182838485868788898A8B8C8D8E8F909192939495969798999A9B9C9D9E9FA0A1A2A3A4A5A6A7A8A9AAABACADAEAFB0B1B2B3B4B5B6B7B8B9BABBBCBDBEBFC0C1C2C3C4C5C6C7C8C9CACBCCCDCECFD0D1D2D3D4D5D6D7D8D9DADBDCDDDEDFE0E1E2E3E4E5E6E7E8E9EAEBECEDEEEFF0F1F2F3F4F5F6F7F8F9FAFBFCFDFEFF00010203	e834297d45be0ae3d42422c3071c57590c65759366d7c2e1871d3d4a15f5759f
		Plaintext Akhir	000102030405060708090A0B0C0D0E0F101112131415161718191A1B1C1D1E1F202122232425262728292A2B2C2D2E2F303132333435363738393A3B3C3D3E3F404142434445464748494A4B4C4D4E4F505152535455565758595A5B5C5D5E5F606162636465666768696A6B6C6D6E6F707172737475767778797A7B7C7D7E7F808182838485868788898A8B8C8D8E8F909192939495969798999A9B9C9D9E9FA0A1A2A3A4A5A6A7A8A9AAABACADAEAFB0B1B2B3B4B5B6B7B8B9BABBBCBDBEBFC0C1C2C3C4C5C6C7C8C9CACBCCCDCECFD0D1D2D3D4D5D6D7D8D9DADBDCDDDEDFE0E1E2E3E4E5E6E7E8E9EAEBECEDEEEFF0F1F2F3F4F5F6F7F8F	e834297d45be0ae3d42422c3071c57590c65759366d7c2e1871d3d4a15f5759f

			9FAFBFCFDFEFF00010203	
	CBC	Plaintext Awal	000102030405060708090A0B0C0D0E0F101112131415161718191A1B1C1D1E1F202122232425262728292A2B2C2D2E2F303132333435363738393A3B3C3D3E3F404142434445464748494A4B4C4D4E4F505152535455565758595A5B5C5D5E5F606162636465666768696A6B6C6D6E6F707172737475767778797A7B7C7D7E7F808182838485868788898A8B8C8D8E8F909192939495969798999A9B9C9D9E9FA0A1A2A3A4A5A6A7A8A9AAABACADAEAFB0B1B2B3B4B5B6B7B8B9BABBBCBDBEBFC0C1C2C3C4C5C6C7C8C9CACBCCCDCECFD0D1D2D3D4D5D6D7D8D9DADBDCDDDEDFE0E1E2E3E4E5E6E7E8E9EAEBECEDEEEFF0F1F2F3F4F5F6F7F8F9FAFBFCFDFEFF00010203	e834297d45be0ae3d42422c3071c57590c65759366d7c2e1871d3d4a15f5759f
		Plaintext Akhir	000102030405060708090A0B0C0D0E0F101112131415161718191A1B1C1D1E1F202122232425262728292A2B2C2D2E2F303132333435363738393A3B3C3D3E3F404142434445464748494A4B4C4D4E4F505152535455565758595A5B5C5D5E5F606162636465666768696A6B6C6D6E6F707172737475767778797A7B7C7D7E7F808182838485868788898A8B8C8D8E8F909192939495969798999A9B9C9D9E9FA0A1A2A3A4A5A6A7A8A9AAABACADAEAFB0B1B2B3B4B5B6B7B8B9BABBBCBDBEBFC0C1C2C3C4C5C6C7C8C9CACBCCCDCECFD0D1D2D3D4D5D6D7D8D9DADBDCDDDEDFE0E1E2E3E4E5E6E7E8E9EAEBECEDEEEFF0F1F2F3F4F5F6F7F8F9FAFBFCFDFEFF00010203	e834297d45be0ae3d42422c3071c57590c65759366d7c2e1871d3d4a15f5759f
	CFB	Plaintext Awal	000102030405060708090A0B0C0D0E0F101112131415161718191A1B1C1D1E1F202122232425262728292A2B2C2D2E2F303132333435363738393A3B3C3D3E3F404142434445464748494A4B4C4D4E4F505152535455565758595A5B5C5D5E5F606162636465666768696A6B6C6D6E6F707172737475767778797A7B7C7D7E7F808182838485868788898A8B8C8D8E8F909192939495969798999A	e834297d45be0ae3d42422c3071c57590c65759366d7c2e1871d3d4a15f5759f

			9B9C9D9E9FA0A1A2A3A4A5A6A7A8 A9AAABACADAEAFB0B1B2B3B4B5 B6B7B8B9BABBBCBDBEBFC0C1C2 C3C4C5C6C7C8C9CACBCCCDCECF D0D1D2D3D4D5D6D7D8D9DADBDC DDDEDFE0E1E2E3E4E5E6E7E8E9EA EBECEDEEEFF0F1F2F3F4F5F6F7F8F 9FAFBFCFDFEFF00010203	
		Plaintext Akhir	000102030405060708090A0B0C0D0E0 F101112131415161718191A1B1C1D1E 1F202122232425262728292A2B2C2D2 E2F303132333435363738393A3B3C3D 3E3F404142434445464748494A4B4C4 D4E4F505152535455565758595A5B5C 5D5E5F606162636465666768696A6B6 C6D6E6F707172737475767778797A7B 7C7D7E7F808182838485868788898A8 B8C8D8E8F909192939495969798999A 9B9C9D9E9FA0A1A2A3A4A5A6A7A8 A9AAABACADAEAFB0B1B2B3B4B5 B6B7B8B9BABBBCBDBEBFC0C1C2 C3C4C5C6C7C8C9CACBCCCDCECF D0D1D2D3D4D5D6D7D8D9DADBDC DDDEDFE0E1E2E3E4E5E6E7E8E9EA EBECEDEEEFF0F1F2F3F4F5F6F7F8F 9FAFBFCFDFEFF00010203	e834297d45be0ae3d42422c 3071c57590c65759366d7c2 e1871d3d4a15f5759f
	OFB	Plaintext Awal	000102030405060708090A0B0C0D0E0 F101112131415161718191A1B1C1D1E 1F202122232425262728292A2B2C2D2 E2F303132333435363738393A3B3C3D 3E3F404142434445464748494A4B4C4 D4E4F505152535455565758595A5B5C 5D5E5F606162636465666768696A6B6 C6D6E6F707172737475767778797A7B 7C7D7E7F808182838485868788898A8 B8C8D8E8F909192939495969798999A 9B9C9D9E9FA0A1A2A3A4A5A6A7A8 A9AAABACADAEAFB0B1B2B3B4B5 B6B7B8B9BABBBCBDBEBFC0C1C2 C3C4C5C6C7C8C9CACBCCCDCECF D0D1D2D3D4D5D6D7D8D9DADBDC DDDEDFE0E1E2E3E4E5E6E7E8E9EA EBECEDEEEFF0F1F2F3F4F5F6F7F8F 9FAFBFCFDFEFF00010203	e834297d45be0ae3d42422c 3071c57590c65759366d7c2 e1871d3d4a15f5759f
		Plaintext Akhir	000102030405060708090A0B0C0D0E0 F101112131415161718191A1B1C1D1E 1F202122232425262728292A2B2C2D2	e834297d45be0ae3d42422c 3071c57590c65759366d7c2 e1871d3d4a15f5759f

			E2F303132333435363738393A3B3C3D 3E3F404142434445464748494A4B4C4 D4E4F505152535455565758595A5B5C 5D5E5F606162636465666768696A6B6 C6D6E6F707172737475767778797A7B 7C7D7E7F808182838485868788898A8 B8C8D8E8F909192939495969798999A 9B9C9D9E9FA0A1A2A3A4A5A6A7A8 A9AAABACADAEAFB0B1B2B3B4B5 B6B7B8B9BABBBCBDBEBFC0C1C2 C3C4C5C6C7C8C9CACBCCCDCECF D0D1D2D3D4D5D6D7D8D9DADBDC DDDEDFE0E1E2E3E4E5E6E7E8E9EA EBECEDEEEFF0F1F2F3F4F5F6F7F8F 9FAFBFCFDFEFF00010203	
	CTR	Plaintext Awal	000102030405060708090A0B0C0D0E0 F101112131415161718191A1B1C1D1E 1F202122232425262728292A2B2C2D2 E2F303132333435363738393A3B3C3D 3E3F404142434445464748494A4B4C4 D4E4F505152535455565758595A5B5C 5D5E5F606162636465666768696A6B6 C6D6E6F707172737475767778797A7B 7C7D7E7F808182838485868788898A8 B8C8D8E8F909192939495969798999A 9B9C9D9E9FA0A1A2A3A4A5A6A7A8 A9AAABACADAEAFB0B1B2B3B4B5 B6B7B8B9BABBBCBDBEBFC0C1C2 C3C4C5C6C7C8C9CACBCCCDCECF D0D1D2D3D4D5D6D7D8D9DADBDC DDDEDFE0E1E2E3E4E5E6E7E8E9EA EBECEDEEEFF0F1F2F3F4F5F6F7F8F 9FAFBFCFDFEFF00010203	e834297d45be0ae3d42422c 3071c57590c65759366d7c2 e1871d3d4a15f5759f
		Plaintext Akhir	000102030405060708090A0B0C0D0E0 F101112131415161718191A1B1C1D1E 1F202122232425262728292A2B2C2D2 E2F303132333435363738393A3B3C3D 3E3F404142434445464748494A4B4C4 D4E4F505152535455565758595A5B5C 5D5E5F606162636465666768696A6B6 C6D6E6F707172737475767778797A7B 7C7D7E7F808182838485868788898A8 B8C8D8E8F909192939495969798999A 9B9C9D9E9FA0A1A2A3A4A5A6A7A8 A9AAABACADAEAFB0B1B2B3B4B5 B6B7B8B9BABBBCBDBEBFC0C1C2 C3C4C5C6C7C8C9CACBCCCDCECF D0D1D2D3D4D5D6D7D8D9DADBDC	e834297d45be0ae3d42422c 3071c57590c65759366d7c2 e1871d3d4a15f5759f

			DDDEDFE0E1E2E3E4E5E6E7E8E9EA EBECEDEEEFF0F1F2F3F4F5F6F7F8F 9FAFBFCFDFEFF00010203	

ANALISIS KEAMANAN

1. Pengujian Keandalan Enkripsi-Dekripsi

Metode

Untuk setiap mode operasi dilakukan proses:

```
ciphertext = Encrypt(plaintext, key, IV)
recovered  = Decrypt(ciphertext, key, IV)
```

Kemudian hasil recovered dibandingkan dengan plaintext awal.
Pengujian dilakukan pada ukuran plaintext: 0, 1, 15, 16, 17, 31, 32, 100, dan 1000 byte

Hasil

Semua mode menghasilkan proses round-trip yang benar:

Mode	Round-trip	Panjang ciphertext
ECB	Berhasil	Kelipatan 16 byte
CBC	Berhasil	Kelipatan 16 byte
CFB	Berhasil	Sama dengan plaintext
OFB	Berhasil	Sama dengan plaintext
CTR	Berhasil	Sama dengan plaintext

Analisis

Hasil ini menunjukkan bahwa implementasi dasar enkripsi dan dekripsi berfungsi dengan benar untuk berbagai ukuran plaintext. ECB dan CBC menggunakan padding PKCS#7 sehingga ciphertext selalu berukuran kelipatan 16 byte. CFB, OFB, dan CTR dapat memproses plaintext dengan panjang tidak kelipatan ukuran blok tanpa menambahkan padding.

2. Analisis Avalanche Effect

Metode

Avalanche Effect mengukur perubahan ciphertext ketika terjadi perubahan kecil pada plaintext. Pada pengujian ini, satu bit plaintext diubah, kemudian plaintext asli dan plaintext yang telah dimodifikasi dienkripsi dengan key dan IV yang sama.

Persentase avalanche dihitung menggunakan:

Avalanche (%) =jumlah bit ciphertext yang berubah / total bit ciphertext × 100%

Nilai ideal secara umum mendekati 50%, yang berarti perubahan satu bit input menyebabkan sekitar setengah bit output berubah.

Hasil

Mode	Bit berubah	Total bit	Avalanche
ECB	262	2432	10,77%
CBC	1189	2432	48,89%
CFB	715	1920	37,24%
OFB	4	1920	0,21%
CTR	4	1920	0,21%
Rata-rata gabungan	2174	10624	20,46%

Analisis

ECB

ECB menghasilkan avalanche sebesar 10,77%. Nilai ini rendah karena setiap blok plaintext diproses secara independen. Perubahan pada satu blok tidak menyebabkan perubahan pada blok lain.

Hal ini sesuai dengan karakteristik ECB. Nilai avalanche ECB tidak hanya merepresentasikan kualitas block cipher internal, tetapi juga menunjukkan bahwa mode ECB tidak memiliki mekanisme chaining antarblok.

CBC

CBC menghasilkan avalanche sebesar 48,89%, paling dekat dengan nilai ideal 50%. Hal ini disebabkan oleh operasi XOR dengan ciphertext blok sebelumnya. Perubahan plaintext tidak hanya memengaruhi blok terkait, tetapi juga dapat memengaruhi proses enkripsi blok berikutnya.

Berdasarkan pengujian ini, CBC menunjukkan difusi antarblok yang lebih baik dibandingkan ECB.

CFB

CFB menghasilkan avalanche sebesar 37,24%. Perubahan plaintext memengaruhi ciphertext melalui mekanisme feedback. Namun, penyebarannya tidak sama dengan CBC karena ciphertext digunakan sebagai input feedback pada proses berikutnya.

OFB dan CTR

OFB dan CTR menghasilkan nilai sangat rendah, yaitu 0,21%. Nilai ini tidak otomatis menunjukkan bahwa algoritma block cipher internal gagal.

Pada OFB dan CTR, keystream dibangkitkan secara independen dari plaintext. Proses dasarnya adalah:

ciphertext = plaintext XOR keystream

Jika satu bit plaintext diubah, maka umumnya hanya bit ciphertext pada posisi yang sama yang berubah. Keystream tidak ikut berubah. Oleh karena itu, nilai avalanche antarbita plaintext memang rendah dan sesuai dengan karakteristik stream-like mode.

3. Analisis Differential

Metode

Pengujian differential dilakukan dengan membuat dua plaintext yang hanya berbeda pada satu byte. Kedua plaintext kemudian dienkripsi menggunakan key dan IV yang sama. Hasil ciphertext dibandingkan menggunakan Hamming distance.

Hasil

Mode	Bit berubah	Total bit	Differential
ECB	6438	64000	10,06%
CBC	22785	64000	35,60%
CFB	10221	51200	19,96%
OFB	100	51200	0,20%
CTR	100	51200	0,20%
Rata-rata gabungan	-	-	14,08%

Analisis

Hasil differential menunjukkan pola yang sama dengan pengujian avalanche:

ECB hanya mengubah ciphertext pada blok yang berkaitan dengan plaintext yang diubah.

CBC menghasilkan perubahan bit terbesar karena adanya chaining.

CFB menghasilkan perubahan melalui mekanisme feedback.

OFB dan CTR hanya mengubah posisi ciphertext yang bersesuaian karena keystream tidak bergantung pada plaintext.

Nilai differential CBC sebesar 35,60% lebih tinggi daripada ECB, tetapi masih belum mencapai 50%. Hal ini menunjukkan bahwa penyebaran perubahan cukup baik pada skenario pengujian, tetapi belum ideal.

4. Analisis Entropi

Metode

Entropi dihitung dari frekuensi kemunculan setiap nilai byte pada ciphertext. Rumus entropi Shannon yang digunakan adalah:

$H(X) = -\Sigma\, p(x) \log2\, p(x)$

Nilai maksimum entropi untuk satu byte adalah 8 bit.

Hasil

Mode	Entropi
ECB	7,9972 bit/byte
CBC	7,9973 bit/byte
CFB	7,9974 bit/byte
OFB	7,9974 bit/byte
CTR	7,9974 bit/byte

Analisis

Semua mode menghasilkan nilai entropi yang sangat dekat dengan nilai maksimum 8 bit/byte. Hal ini menunjukkan bahwa distribusi nilai byte ciphertext pada sampel 64 KB relatif merata dan tidak menunjukkan pola sederhana yang dominan.

Nilai entropi tertinggi terdapat pada CFB, OFB, dan CTR, yaitu sekitar 7,9974 bit/byte. Perbedaannya dengan ECB sangat kecil, sehingga tidak tepat menyimpulkan bahwa satu mode jauh lebih aman hanya berdasarkan perbedaan tersebut.

5. Analisis Histogram

Metode

Histogram digunakan untuk melihat jumlah nilai byte berbeda pada plaintext dan ciphertext. Pengujian menggunakan plaintext berulang yang hanya memiliki satu nilai byte unik, yaitu byte 0x41.

Hasil

Mode	Byte unik plaintext	Byte unik ciphertext
ECB	1	29
CBC	1	256
CFB	1	256
OFB	1	256
CTR	1	256

Analisis

Plaintext pengujian memiliki pola yang sangat jelas karena seluruh byte bernilai sama. Setelah dienkripsi:

ECB menghasilkan 29 byte unik.

CBC, CFB, OFB, dan CTR menghasilkan seluruh 256 kemungkinan nilai byte.

ECB menghasilkan jumlah byte unik yang lebih sedikit dibandingkan mode lainnya. Hal ini menunjukkan bahwa ECB masih memiliki karakteristik distribusi yang lebih terstruktur dibandingkan mode lain.

Selain itu, ECB tidak menggunakan chaining. Blok plaintext yang identik akan menghasilkan blok ciphertext yang identik selama key yang digunakan sama. Hal ini dapat membocorkan pola data.

CBC, CFB, OFB, dan CTR menghasilkan distribusi byte yang lebih beragam pada pengujian ini. Namun, hasil histogram tidak boleh dianggap sebagai pembuktian keamanan menyeluruh karena hasilnya bergantung pada ukuran data, key, IV, dan plaintext yang digunakan.

KESIMPULAN & SARAN

Algoritma block cipher yang dirancang berhasil diimplementasikan. Yang setelah pengetesan terbukti mampu melakukan proses enkripsi dan dekripsi secara benar pada seluruh mode operasi (ECB, CBC, CFB, OFB, dan CTR). Pengujian karakteristik keamanan menunjukkan bahwa mode CBC memberikan difusi antar blok terbaik dengan avalanche effect mendekati nilai ideal (48,89%), sementara seluruh mode operasi menghasilkan nilai entropi Shannon yang sangat tinggi (sekitar 7,997 bit/byte), menandakan distribusi ciphertext yang merata dan acak.

Kemudian untuk kedepannya cipher ini bisa dikembangkan lebih lanjut. Seperti dai penggunaan *Key-Dependent S-Box* dinamis pada efisiensi waktu komputasinya saat inisialisasi kunci. Agar proses pembangkitan awal tidak menjadi hambatan performa pada aplikasi *real-time*. Kemudian pengujian ketahanan keamanan disarankan untuk diperluas mencakup simulasi differential cryptanalysis dan linear cryptanalysis secara mendalam untuk validasi ketahanan cipher terhadap serangan kriptografi tingkat lanjut selain pengujian statistik dasar.

DAFTAR PUSTAKA

- https://informatika.stei.itb.ac.id/~rinaldi.munir/Kriptografi/2026-2027/09-Beberapa-block-cipher-bagian1-2026.pdf
- https://informatika.stei.itb.ac.id/~rinaldi.munir/Kriptografi/2026-2027/08-Block-Cipher-Bagian1-(2026).pdf
- https://informatika.stei.itb.ac.id/~rinaldi.munir/Kriptografi/2026-2027/10-Beberapa-block-cipher-bagian2-2026.pdf
- https://informatika.stei.itb.ac.id/~rinaldi.munir/Kriptografi/2026-2027/11-Beberapa-block-cipher-bagian3-2026.pdf

LAMPIRAN

Pranala Github: https://github.com/Narrr21/Tugas2_Kriptografi
Link API Documentation: https://narrr21.github.io/Tugas2_Kriptografi/
Pembagian Tugas:

Nama	**NIM**	**Tugas**
Samuel Gerrard H. Girsang	13523064	Laporan, Analisis keamanan
Nadhif Al Rozin	13523076	Laporan, Setup project dan rancangan cipher
Ivant Samuel Silaban	13523129	Laporan, Implementasi block cipher
Joel Hotlan Haris Siahaan	13523025	Laporan, GUI